

Brief Python

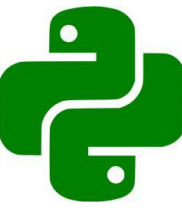
CS 30 v 2.0

First 4-week Python Course
for Beginners

Chapter 4: Data Collections

Dr. Eric Chou, IEEE Senior
Member



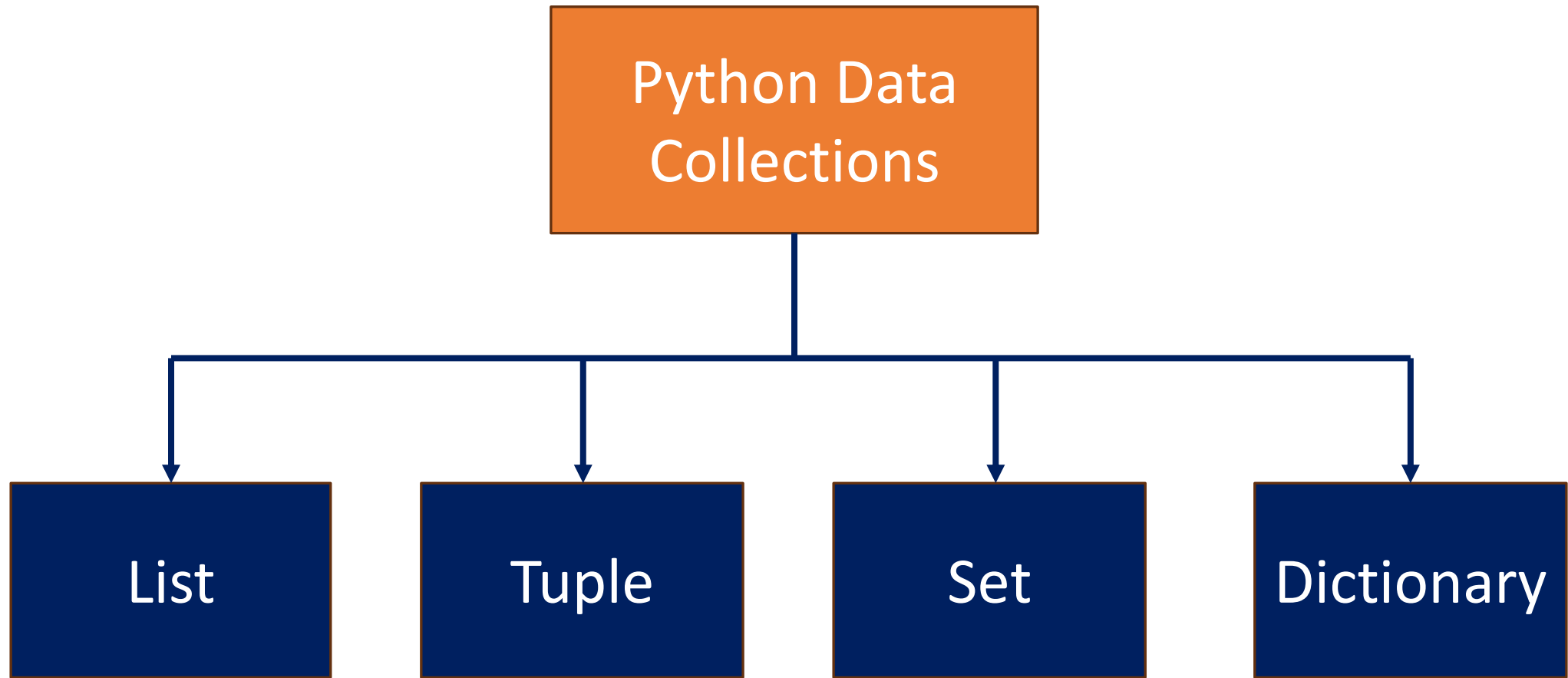


Objectives

- Study the built-in Basic Data Collections in Python
 - Tuple
 - List
 - Set
 - Dictionary

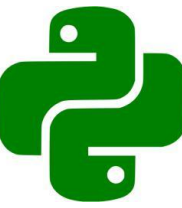
Introduction

LECTURE 1



Tuple

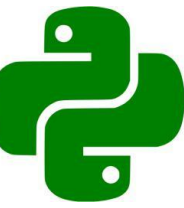
LECTURE 2



Tuples

Same as lists but

- Immutable
- Enclosed in parentheses
- A tuple with a single element ***must*** have a comma inside the parentheses:
 - **`a = (11,)`**



Examples

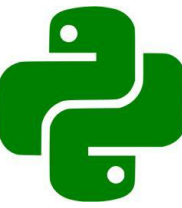
```
>>> mytuple = (11, 22, 33)
```

```
>>> mytuple[0]  
11
```

```
>>> mytuple[-1]  
33
```

```
>>> mytuple[0:1]  
(11, )
```

The comma is required!



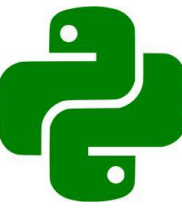
Why?

No confusion possible between **[11]** and **11**

(11) is a perfectly acceptable expression

- **(11) without the comma** is the integer 11
- **(11,) with the comma** is a list containing the integer 11

Sole dirty trick played on us by tuples!



Tuples are immutable

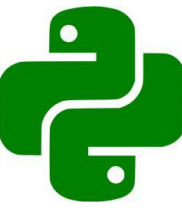
```
>>> mytuple = (11, 22, 33)
```

```
>>> saved = mytuple
```

```
>>> mytuple += (44,)
```

```
>>> mytuple  
(11, 22, 33, 44)
```

```
>>> saved  
(11, 22, 33)
```



Things that do not work

```
mytuple += 55
```

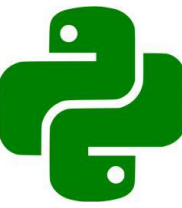
```
Traceback (most recent call last):
```

```
...
```

```
TypeError:
```

```
    can only concatenate tuple (not "int")  
to tuple
```

- Can understand that!



Sorting tuples

```
>>> atuple = (33, 22, 11)
```

```
>>> atuple.sort()
```

```
Traceback (most recent call last):
```

```
...
```

```
AttributeError:
```

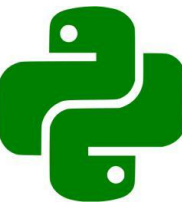
```
'tuple' object has no attribute 'sort'
```

```
>>> atuple = sorted(atuple)
```

```
>>> atuple  
[11, 22, 33]
```

Tuples are immutable!

sorted() returns a list!



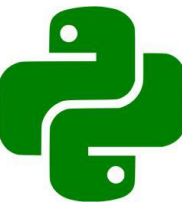
Most other things work!

```
>>> atuple = (11, 22, 33)
```

```
>>> len(atuple)  
3
```

```
>>> 44 in atuple  
False
```

```
>>> [ i for i in atuple]  
[11, 22, 33]
```



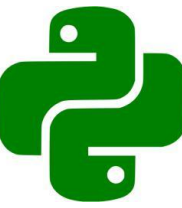
The reverse does not work

```
>>> alist = [11, 22, 33]
```

```
>>> (i for i in alist)
```

```
<generator object <genexpr> at 0x02855DA0>
```

Does not work!



Converting sequences into tuples

```
>>> alist = [11, 22, 33]
```

```
>>> atuple = tuple(alist)
```

```
>>> atuple  
(11, 22, 33)
```

```
>>> newtuple = tuple('Hello World!')
```

```
>>> newtuple  
( 'H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd', '!')
```

Python Tuple Functions

len()

max()

min()

sum()

any()

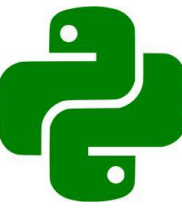
all()

sorted()

tuple()

List

LECTURE 3

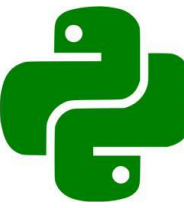


A List is a Kind of Collection

- A collection allows us to put many values in a single “variable”
- A collection is nice because we can carry all many values around in one convenient package.

```
friends = [ 'Joseph', 'Glenn', 'Sally' ]
```

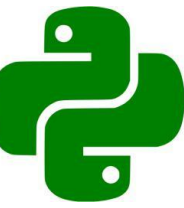
```
carryon = [ 'socks', 'shirt', 'perfume' ]
```



List Constants

- List constants are surrounded by square brackets and the elements in the list are separated by commas

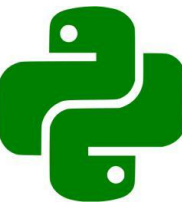
```
>>> print([1, 24, 76])
[1, 24, 76]
>>> print(['red', 'yellow', 'blue'])
['red', 'yellow', 'blue']
>>> print(['red', 24, 98.6])
['red', 24, 98.6]
>>> print([ 1, [5, 6], 7])
[1, [5, 6], 7]
>>> print([])
[]
```
- A list element can be any Python object - even another list
- A list can be empty



We Already Use Lists!

```
for i in [5, 4, 3, 2, 1] :  
    print(i)  
print('Blastoff!')
```

5
4
3
2
1
Blastoff!

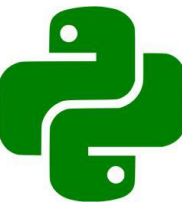


Lists and Definite Loops - Best Pals

```
friends = ['Joseph', 'Glenn', 'Sally']  
for friend in friends :  
    print('Happy New Year:', friend)  
print('Done!')
```

Happy New Year: Joseph
Happy New Year: Glenn
Happy New Year: Sally
Done!

```
z = ['Joseph', 'Glenn', 'Sally']  
for x in z:  
    print('Happy New Year:', x)  
print('Done!')
```

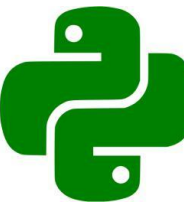


Looking Inside Lists

- Just like strings, we can get at any single element in a list using an index specified in square brackets

Joseph	Glenn	Sally
0	1	2

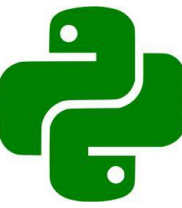
```
>>> friends = [ 'Joseph', 'Glenn', 'Sally' ]  
>>> print(friends[1])  
Glenn  
>>>
```



Lists are Mutable

- Strings are “immutable” - we cannot change the contents of a string - we must make a new string to make any change
- Lists are “mutable” - we can change an element of a list using the index operator

```
>>> fruit = 'Banana'
>>> fruit[0] = 'b'
Traceback
TypeError: 'str' object does not
support item assignment
>>> x = fruit.lower()
>>> print(x)
banana
>>> lotto = [2, 14, 26, 41, 63]
>>> print(lotto)
[2, 14, 26, 41, 63]
>>> lotto[2] = 28
>>> print(lotto)
[2, 14, 28, 41, 63]
```

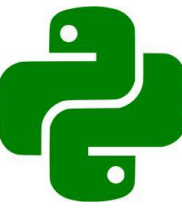


How Long is a List?

- The `len()` function takes a list as a parameter and returns the number of elements in the list
- Actually `len()` tells us the number of elements of any set or sequence (such as a string...)

```
>>> greet = 'Hello Bob'
>>> print(len(greet))
9
>>> x = [ 1, 2, 'joe', 99]
>>> print(len(x))
4
>>>
```

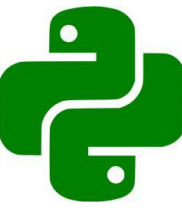
List Functions	Meanings
<code>list.append(x)</code>	Appends object x to list
<code>list.count(x)</code>	Returns count of how many times x occurs in list
<code>list.remove(x)</code>	Removes xect x from list
<code>list.reverse()</code>	Reverses objects of list in place
<code>list.extend(seq)</code>	Appends the contents of seq to list
<code>list.index(x)</code>	Returns the lowest index in list that x appears
<code>list.insert(index, x)</code>	Inserts xect x into list at offset index
<code>list.pop(x=list[-1])</code>	Removes and returns last object or x from list



Using the range Function

- The range function returns a list of numbers that range from zero to one less than the parameter
- We can construct an index loop using for and an integer iterator

```
>>> print(range(4))  
[0, 1, 2, 3]  
>>> friends = ['Joseph', 'Glenn', 'Sally']  
>>> print(len(friends))  
3  
>>> print(range(len(friends)))  
[0, 1, 2]  
>>>
```



A Tale of Two Loops...

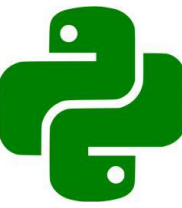
```
friends = ['Joseph', 'Glenn', 'Sally']

for friend in friends :
    print('Happy New Year:', friend)

for i in range(len(friends)) :
    friend = friends[i]
    print('Happy New Year:', friend)
```

```
>>> friends = ['Joseph', 'Glenn', 'Sally']
>>> print(len(friends))
3
>>> print(range(len(friends)))
[0, 1, 2]
>>>
```

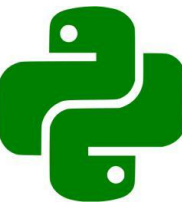
```
Happy New Year: Joseph
Happy New Year: Glenn
Happy New Year: Sally
```



Concatenating Lists Using +

- We can create a new list by adding two existing lists together

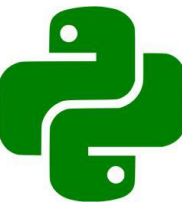
```
>>> a = [1, 2, 3]
>>> b = [4, 5, 6]
>>> c = a + b
>>> print(c)
[1, 2, 3, 4, 5, 6]
>>> print(a)
[1, 2, 3]
```



Lists Can Be Sliced Using :

```
>>> t = [9, 41, 12, 3, 74, 15]
>>> t[1:3]
[41, 12]
>>> t[:4]
[9, 41, 12, 3]
>>> t[3:]
[3, 74, 15]
>>> t[:]
[9, 41, 12, 3, 74, 15]
```

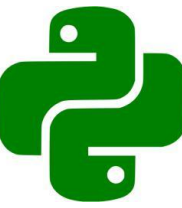
Remember: Just like in strings, the second number is “up to but not including”



List Methods

```
>>> x = list()
>>> type(x)
<type 'list'>
>>> dir(x)
['append', 'count', 'extend', 'index', 'insert',
'pop', 'remove', 'reverse', 'sort']
>>>
```

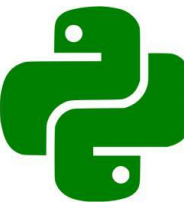
<http://docs.python.org/tutorial/datastructures.html>



Building a List from Scratch

- We can create an empty list and then add elements using the append method
- The list stays in order and new elements are added at the end of the list

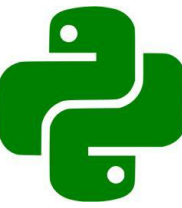
```
>>> stuff = list()
>>> stuff.append('book')
>>> stuff.append(99)
>>> print(stuff)
['book', 99]
>>> stuff.append('cookie')
>>> print(stuff)
['book', 99, 'cookie']
```



Is Something in a List?

- Python provides two operators that let you check if an item is in a list
- These are logical operators that return True or False
- They do not modify the list

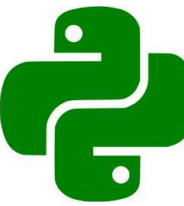
```
>>> some = [1, 9, 21, 10, 16]
>>> 9 in some
True
>>> 15 in some
False
>>> 20 not in some
True
>>>
```



Lists are in Order

- A list can hold many items and keeps those items in the order until we do something to change the order
- A list can be sorted (i.e., change its order)
- The sort method (unlike in strings) means “sort yourself”

```
>>> friends = [ 'Joseph', 'Glenn', 'Sally' ]
>>> friends.sort()
>>> print(friends)
['Glenn', 'Joseph', 'Sally']
>>> print(friends[1])
Joseph
>>>
```

Built-in Functions and Lists

- There are a number of functions built into Python that take lists as parameters
- Remember the loops we built? These are much simpler.

```
>>> nums = [3, 41, 12, 9, 74, 15]
>>> print(len(nums))
6
>>> print(max(nums))
74
>>> print(min(nums))
3
>>> print(sum(nums))
154
>>> print(sum(nums)/len(nums))
25.6
```

Algorithm 1:

```
total = 0
count = 0
while True :
    inp = input('Enter a number: ')
    if inp == 'done' : break
    value = float(inp)
    total = total + value
    count = count + 1

average = total / count
print('Average:', average)
```

Enter a number: 3

Enter a number: 9

Enter a number: 5

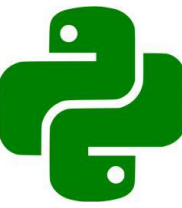
Enter a number: done

Average: 5.666666666667

Algorithm 2:

```
numlist = list()
while True :
    inp = input('Enter a number: ')
    if inp == 'done' : break
    value = float(inp)
    numlist.append(value)

average = sum(numlist) / len(numlist)
print('Average:', average)
```



Best Friends: Strings and Lists

```
>>> abc = 'With three words'
>>> stuff = abc.split()
>>> print(stuff)
['With', 'three', 'words']
>>> print(len(stuff))
3
>>> print(stuff[0])
With

>>> print(stuff)
['With', 'three', 'words']
>>> for w in stuff :
...     print(w)
...
With
Three
Words
>>>
```

Split breaks a string into parts and produces a list of strings. We think of these as words. We can access a particular word or loop through all the words.

Example:

- When you do not specify a delimiter, multiple spaces are treated like one delimiter
- You can specify what delimiter character to use in the splitting

```
>>> line = 'A lot of spaces'
>>> etc = line.split()
>>> print(etc)
['A', 'lot', 'of', 'spaces']
>>>
>>> line = 'first;second;third'
>>> thing = line.split()
>>> print(thing)
['first;second;third']
>>> print(len(thing))
1
>>> thing = line.split(';')
>>> print(thing)
['first', 'second', 'third']
>>> print(len(thing))
3
>>>
```

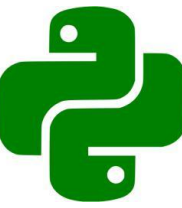
Example:

From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008

```
fhand = open('mbox-short.txt')
for line in fhand:
    line = line.rstrip()
    if not line.startswith('From ') : continue
    words = line.split()
    print(words[2])
```

Sat
Fri
Fri
Fri
...

```
>>> line = 'From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008'
>>> words = line.split()
>>> print(words)
['From', 'stephen.marquard@uct.ac.za', 'Sat', 'Jan', '5', '09:14:16', '2008']
>>>
```

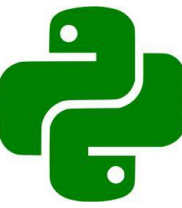


The Double Split Pattern

Sometimes we split a line one way, and then grab one of the pieces of the line and split that piece again

```
From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
```

```
words = line.split()  
email = words[1]  
print pieces[1]
```

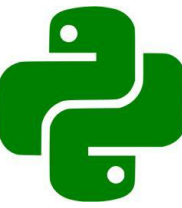


The Double Split Pattern

From **stephen.marquard@uct.ac.za** Sat Jan 5 09:14:16 2008

```
words = line.split()  
email = words[1]  
print pieces[1]
```

```
stephen.marquard@uct.ac.za
```

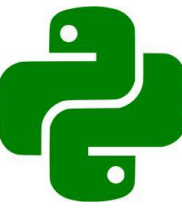


The Double Split Pattern

From **stephen.marquard@uct.ac.za** Sat Jan 5 09:14:16 2008

```
words = line.split()
email = words[1]
pieces = email.split('@')
print pieces[1]
```

```
stephen.marquard@uct.ac.za
['stephen.marquard', 'uct.ac.za']
```

The Double Split Pattern

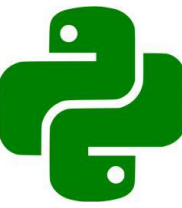
From **stephen.marquard@uct.ac.za** Sat Jan 5 09:14:16 2008

```
words = line.split()
email = words[1]
pieces = email.split('@')
print(pieces[1])
```

```
stephen.marquard@uct.ac.za
['stephen.marquard', 'uct.ac.za']
'uct.ac.za'
```

Set

LECTURE 4



Sets

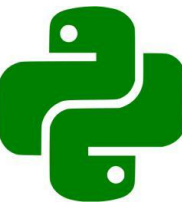
Identified by *curly braces*

- {'Alice', 'Bob', 'Carol'}
- {'Dean'} is a *singleton*

Can only contain *unique elements*

- *Duplicates are eliminated*

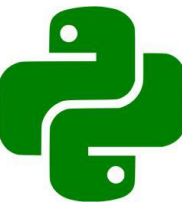
Immutable like tuples and strings



Sets do not contain duplicates

```
>>> cset = {11, 11, 22}
```

```
>>> cset  
{11, 22}
```



Sets are immutable

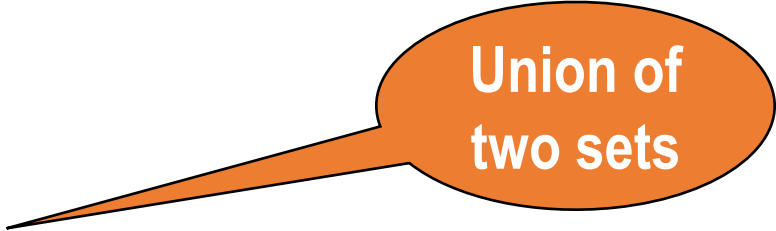
```
>>> aset = {11, 22, 33}
```

```
>>> bset = aset
```

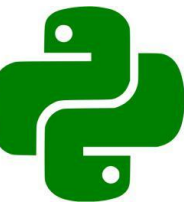
```
>>> aset = aset | {55}
```

```
>>> aset  
{33, 11, 22, 55}
```

```
>>> bset  
{33, 11, 22}
```



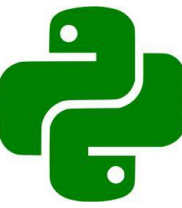
Union of
two sets



Sets have no order

```
>>> {1, 2, 3, 4, 5, 6, 7}
{1, 2, 3, 4, 5, 6, 7}
```

```
>>> {11, 22, 33}
{33, 11, 22}
```



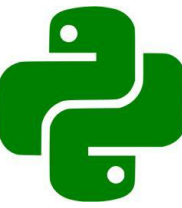
Sets do not support indexing

```
>>> myset = {'Apples', 'Bananas', 'Oranges'}
```

```
>>> myset  
{'Bananas', 'Oranges', 'Apples'}
```

```
>>> myset[0]
```

```
Traceback (most recent call last):  
  File "<pyshell#2>", line 1, in <module>  
    myset[0]  
TypeError: 'set' object does not support  
indexing
```



Examples

```
>>> alist = [11, 22, 33, 22, 44]
```

```
>>> aset = set(alist)
```

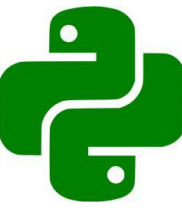
```
>>> aset
```

```
{33, 11, 44, 22}
```

```
>>> aset = aset + {55}
```

```
SyntaxError: invalid syntax
```


Operation	Equivalent	Result
<code>len(s)</code>		number of elements in set <i>s</i> (cardinality)
<code>x in s</code>		test <i>x</i> for membership in <i>s</i>
<code>x not in s</code>		test <i>x</i> for non-membership in <i>s</i>
<code>s.issubset(t)</code>	<code>s <= t</code>	test whether every element in <i>s</i> is in <i>t</i>
<code>s.issuperset(t)</code>	<code>s >= t</code>	test whether every element in <i>t</i> is in <i>s</i>
<code>s.union(t)</code>	<code>s t</code>	new set with elements from both <i>s</i> and <i>t</i>
<code>s.intersection(t)</code>	<code>s & t</code>	new set with elements common to <i>s</i> and <i>t</i>
<code>s.difference(t)</code>	<code>s - t</code>	new set with elements in <i>s</i> but not in <i>t</i>
<code>s.symmetric_difference(t)</code>	<code>s ^ t</code>	new set with elements in either <i>s</i> or <i>t</i> but not both
<code>s.copy()</code>		new set with a shallow copy of <i>s</i>



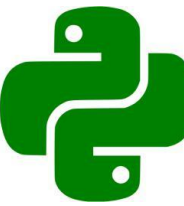
Boolean operations on sets (I)

Union of two sets



Contains all elements that are in set **A** or in set **B**



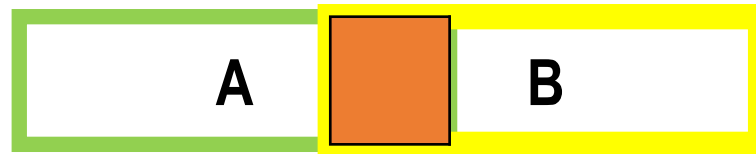


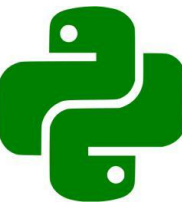
Boolean operations on sets (II)

Intersection of two sets



Contains all elements that are in both sets **A** and **B**





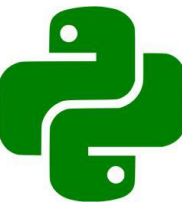
Boolean operations on sets (III)

Difference of two sets



Contains all elements that are in A but not in B





Boolean operations on sets (IV)

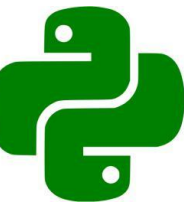
Symmetric difference of two sets



Contains all elements that are either

- in set **A** but not in set **B** or
- in set **B** but not in set **A**





Boolean operations on sets (V)

```
>>> aset = {11, 22, 33}
```

```
>>> bset = {12, 23, 33}
```

Union of two sets

- ```
>>> aset | bset
```

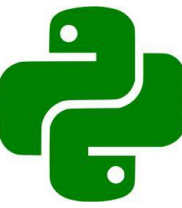
```
{33, 22, 23, 11, 12}
```

## Intersection of two sets:

- ```
>>> aset & bset
```



```
{33}
```



Boolean operations on sets (VI)

```
>>> aset = {11, 22, 33}
```

```
>>> bset = {12, 23, 33}
```

Difference:

- ```
>>> aset - bset
```

```
{11, 22}
```

## Symmetric difference:

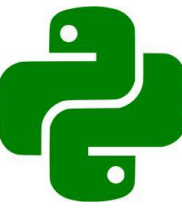
- ```
>>> aset ^ bset
```



```
{11, 12, 22, 23}
```

Dictionary

LECTURE 5



Dictionaries (I)

Store *pairs* of entries called *items*

```
{ 'CS' : '743-713-3350', 'UHPD' : '713-743-3333' }
```

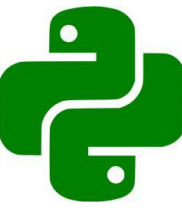
Each pair of entries contains

- A *key*
- A *value*

Key and values are separated by a colon

Pair of entries are separated by commas

Dictionary is enclosed within curly braces



Usage

Keys must be *unique* within a dictionary

- No *duplicates*

If we have

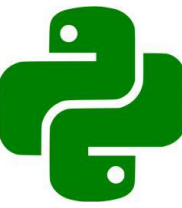
age = {'Alice' : 25, 'Bob' :28}

then

age['Alice'] is 25

and

age[Bob'] is 28



Dictionaries are mutable

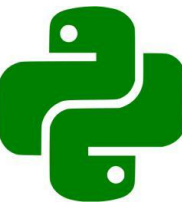
```
>>> age = {'Alice' : 25, 'Bob' : 28}
```

```
>>> saved = age
```

```
>>> age['Bob'] = 29
```

```
>>> age  
{'Bob': 29, 'Alice': 25}
```

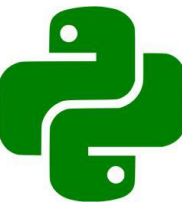
```
>>> saved  
{'Bob': 29, 'Alice': 25}
```



Keys must be unique

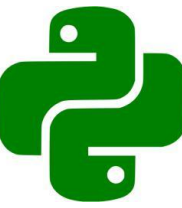
```
>>> age = {'Alice' : 25, 'Bob' : 28, 'Alice' : 26}
```

```
>>> age  
{'Bob': 28, 'Alice': 26}
```



Displaying contents

```
>>> age = {'Alice' : 25, 'Carol': 'twenty-two'}  
  
>>> age.items()  
dict_items([ ('Alice', 25), ('Carol', 'twenty-two')])  
  
>>> age.keys()  
dict_keys([ 'Alice', 'Carol'])  
  
age.values()  
dict_values([28, 25, 'twenty-two'])
```



Updating directories

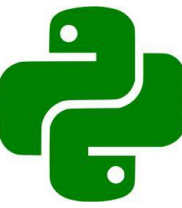
```
>>> age = {'Alice': 26 , 'Carol' : 22}
```

```
>>> age.update({'Bob' : 29})
```

```
>>> age  
{'Bob': 29, 'Carol': 22, 'Alice': 26}
```

```
>>> age.update({'Carol' : 23})
```

```
>>> age  
{'Bob': 29, 'Carol': 23, 'Alice': 26}
```

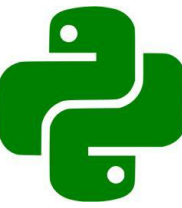


Returning a value

```
>>> age = {'Bob': 29, 'Carol': 23, 'Alice': 26}
```

```
>>> age.get('Bob')  
29
```

```
>>> age['Bob']  
29
```



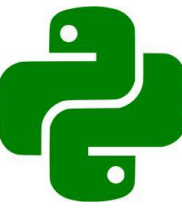
Removing a specific item (I)

```
>>> a = {'Alice' : 26, 'Carol' : 'twenty-two'}
```

```
>>> a  
{'Carol': 'twenty-two', 'Alice': 26}
```

```
>>> a.pop('Carol')  
'twenty-two'
```

```
>>> a  
{'Alice': 26}
```

Removing a specific item (II)

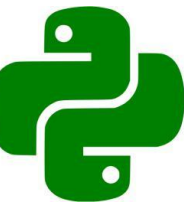
```
>>> a.pop('Alice')
```

```
26
```

```
>>> a
```

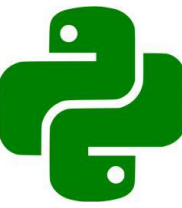
```
{}
```

```
>>>
```



Remove a random item

```
>>> age = {'Bob': 29, 'Carol': 23, 'Alice': 26}
>>> age.popitem()
('Bob', 29)
>>> age
{'Carol': 23, 'Alice': 26}
>>> age.popitem()
('Carol', 23)
>>> age
{'Alice': 26}
```

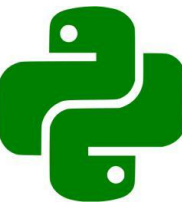


Summary

Strings, lists, tuples, sets and dictionaries all deal with aggregates

Two big differences

- ***Lists*** and ***dictionaries*** are ***mutable***
 - Unlike strings, tuples and sets
- ***Strings, lists*** and ***tuples*** are ***ordered***
 - Unlike sets and dictionaries



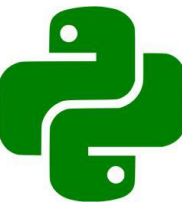
Mutable aggregates

Can modify individual items

- **x = [11, 22, 33]**
x[0] = 44
will work

Cannot save current value

- **x = [11, 22, 33]**
y = x
will **not** work



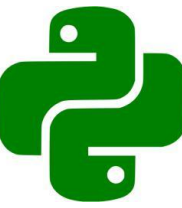
Immutable aggregates

Cannot modify individual items

- **s = 'hello!'**
 s[0] = 'H'
 is an **ERROR**

Can save current value

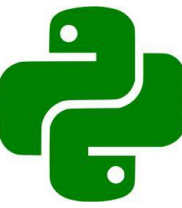
- **s = 'hello!'**
 t = s
 will work



Ordered aggregates

Entities in the collection can be accessed through a **numerical index**

- `s = 'Hello!'`
`s[0]`
- `x = ['Alice', 'Bob', 'Carol']`
`x[-1]`
- `t = (11, 22)`
`t[1]`



Other aggregates

Cannot index sets

- `myset = {'Apples', 'Bananas', 'Oranges'}` `myset[0]` is **WRONG**

Can only index dictionaries **through their keys**

- `age = {'Bob': 29, 'Carol': 23, 'Alice': 26}`
`age['Alice']` works
`age[0]` is **WRONG**

Python Dictionary Methods

Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and values
get()	Returns the value of the specified key
items()	Returns a list containing the a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary

JSON	Python
object	dict
array	list
string	unicode
number (int)	int, long
number (real)	float
TRUE	TRUE
FALSE	FALSE
null	None